

A Quick Tour of Python

Dr. Sumit Kumar Tetarave

ML Lab #1: Part 5

File I/O

- Files are persistent storage
- Allow data to be stored beyond program lifetime
- The basic operations on files are
 - open, close, read, write
- Python treat files as sequence of lines
 - sequence operations work for the data read from files

File I/O: **open** and **close**

open(filename, mode)

- While opening a file, you need to supply
 - The name of the file, including the path
 - The mode in which you want to open a file
 - Common modes are **r** (read), **w** (write), **a** (append)
- Mode is optional, defaults to **r**
- **open**(..) returns a file object
- **close**() on the file object closes the file
 - finishes any buffered operations

File I/O: Example

```
>>> players = open('tennis_players', 'w')
>>>
>>> • Do some writing
>>> • How to do it?
>>>   • see the next few slides
>>>
>>> players.close() # done with writing
```

File I/O: **read**, **write** and **append**

- Reading from an open file returns the contents of the file
 - as **sequence** of lines in the program
- Writing to a file
 - **IMPORTANT:** If opened with mode '**w**', **clears** the existing contents of the file
 - Use append mode ('**a**') to preserve the contents
 - Writing happens at the end

File I/O: Examples

```
>>> players = open('tennis_players', 'w')
>>> players.write('Roger Federar\n')
>>> players.write('Rafael Nadal\n')
>>> players.write('Andy Murray\n')
>>> players.write('Novak Djokovic\n')
>>> players.write('Leander Paes\n')
>>> players.close() # done with writing

>>> countries = open('tennis_countries', 'w')
>>> countries.write('Switzerland\n')
>>> countries.write('Spain\n')
>>> countries.write('Britain\n')
>>> countries.write('Serbia\n')
>>> countries.write('India\n')

>>> countries.close() # done with writing
```

File I/O: Examples

```
>>> print(players)
<closed file 'tennis_players', mode 'w' at 0x031A48B8>
>>> print(countries)
<closed file 'tennis_countries', mode 'w' at 0x031A49C0>

>>> n = open('tennis_players', 'r')
>>> c = open('tennis_countries', 'r')
>>> n
<open file 'tennis_players', mode 'r' at 0x031A4910>
>>> c
<open file 'tennis_countries', mode 'r' at 0x031A4A70>
```

```
>>> pn = n.read() # read all players
>>> pn
'Roger Federar\nRafael Nadal\nAndy Murray\nNovak Djokovic\nLeander Paes\n'
>>> print(pn)
Roger Federar
Rafael Nadal
Andy Murray
Novak Djokovic
Leander Paes
```



Note empty line due to '\n'

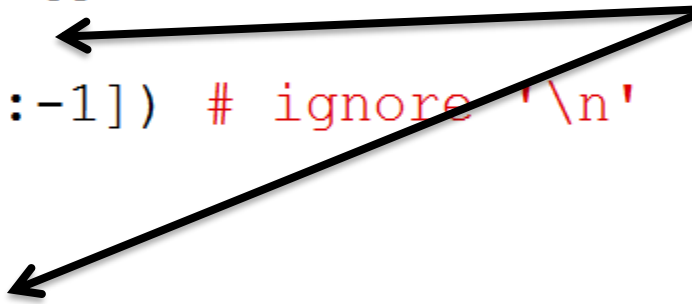
```
>>> |
>>> n.close()
```


File I/O: Examples

```
>>> n = open('tennis_players', 'r')
>>> c = open('tennis_countries', 'r')
>>> pn, pc = [], []
>>> for l in n:
    pn.append(l[:-1]) # ignore '\n'
>>> n.close()
>>> for l in c:
    pc.append(l[:-1])
>>> c.close()

>>> print(pn, '\n', pc)
['Roger Federar', 'Rafael Nadal', 'Andy Murra
y', 'Novak Djokovic', 'Leander Paes']
['Switzerland', 'Spain', 'Britain', 'Serbia',
'India']
```

Note the use of for ... in
for sequence



File I/O: Examples

```
>>> name_country = []
>>> for i in range(len(pn)):
    name_country.append((pn[i], pc[i]))

>>> print(name_country)
[('Roger Federar', 'Switzerland'), ('Rafael N
adal', 'Spain'), ('Andy Murray', 'Britain'),
('Novak Djokovic', 'Serbia'), ('Leander Paes'
, 'India')]
>>> n2c = dict(name_country)
>>> print(n2c)
{'Roger Federar': 'Switzerland', 'Andy Murray
': 'Britain', 'Leander Paes': 'India', 'Novak
Djokovic': 'Serbia', 'Rafael Nadal': 'Spain'}
>>> print(n2c['Leander Paes'])
India
```

Understanding Search Algorithms

Understanding Search using 8-puzzle

1	2	4
5		7
3	6	8

Initial state

1	4	7
2	5	8
3	6	

final state

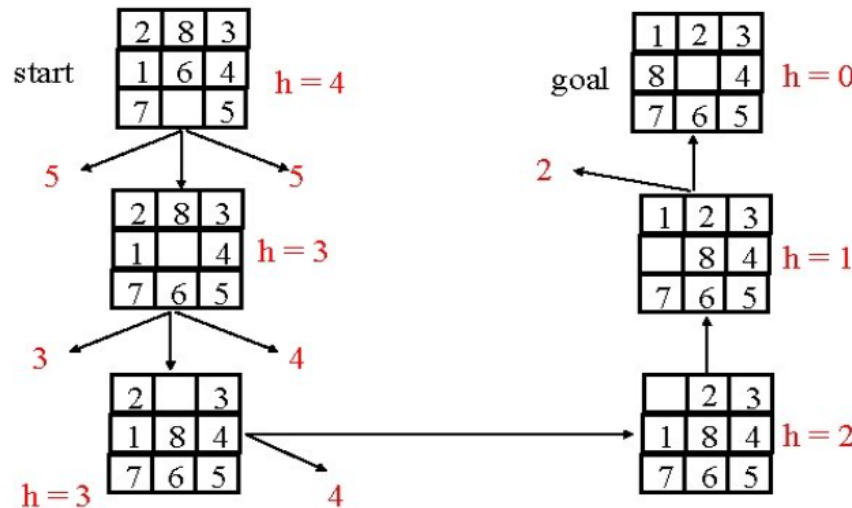
h_1 = the number of misplaced tiles.

h_1 is an admissible heuristic because it is clear that any tile that is out of place must be moved at least once.

Hill-climbing search Conclusion

- The algorithm does not maintain a search tree,
 - So, the data structure for the current node need only record the state and the value of the objective function.
- Hill climbing does not look ahead beyond the immediate neighbors of the current state.

Hill climbing example



$f(n) = (\text{number of tiles out of place})$

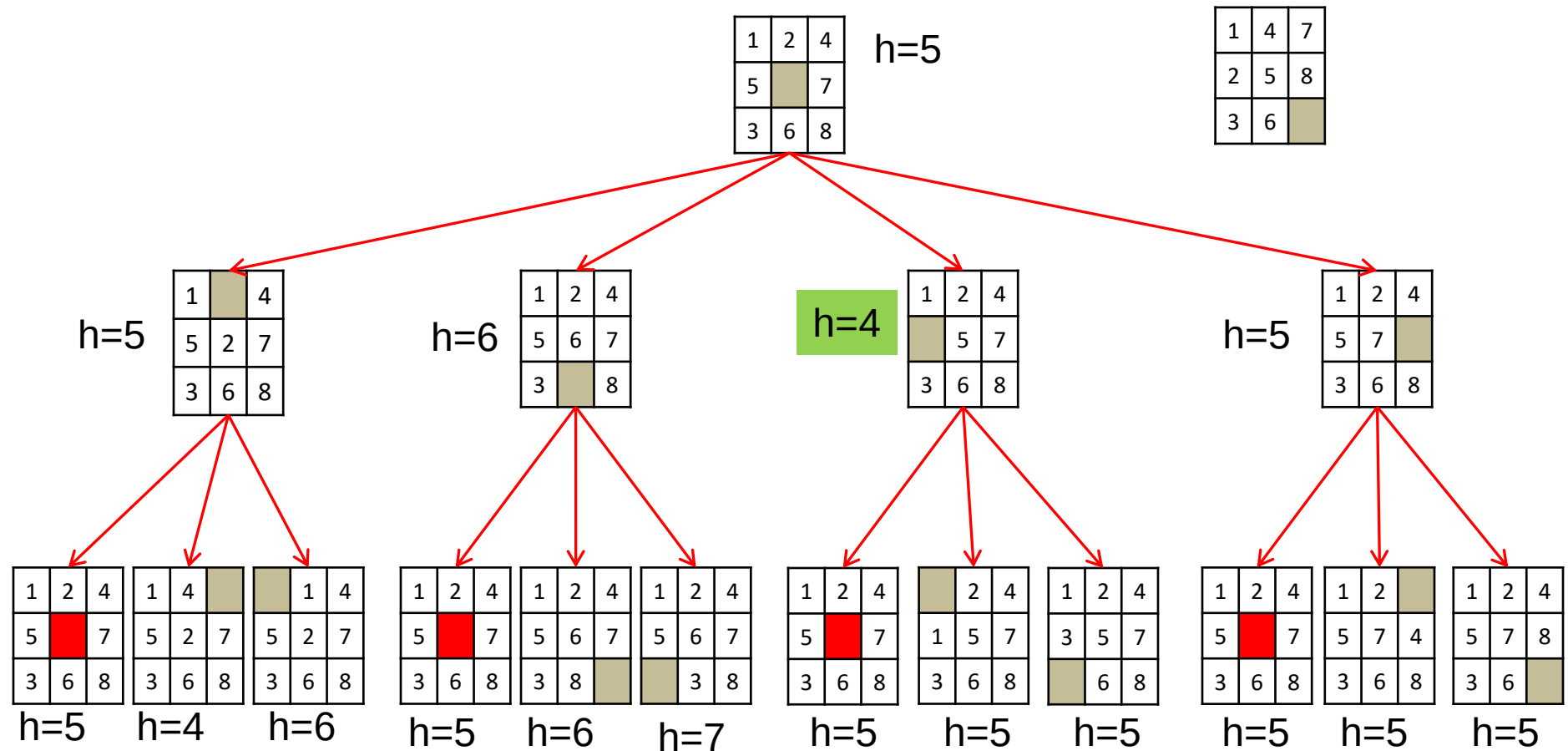
Properties of greedy best-first search

- Complete? No – can get stuck in loops, e.g.,
lasi $\rightarrow$ Neamt $\rightarrow$ lasi $\rightarrow$ Neamt $\rightarrow$
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

Properties of A*

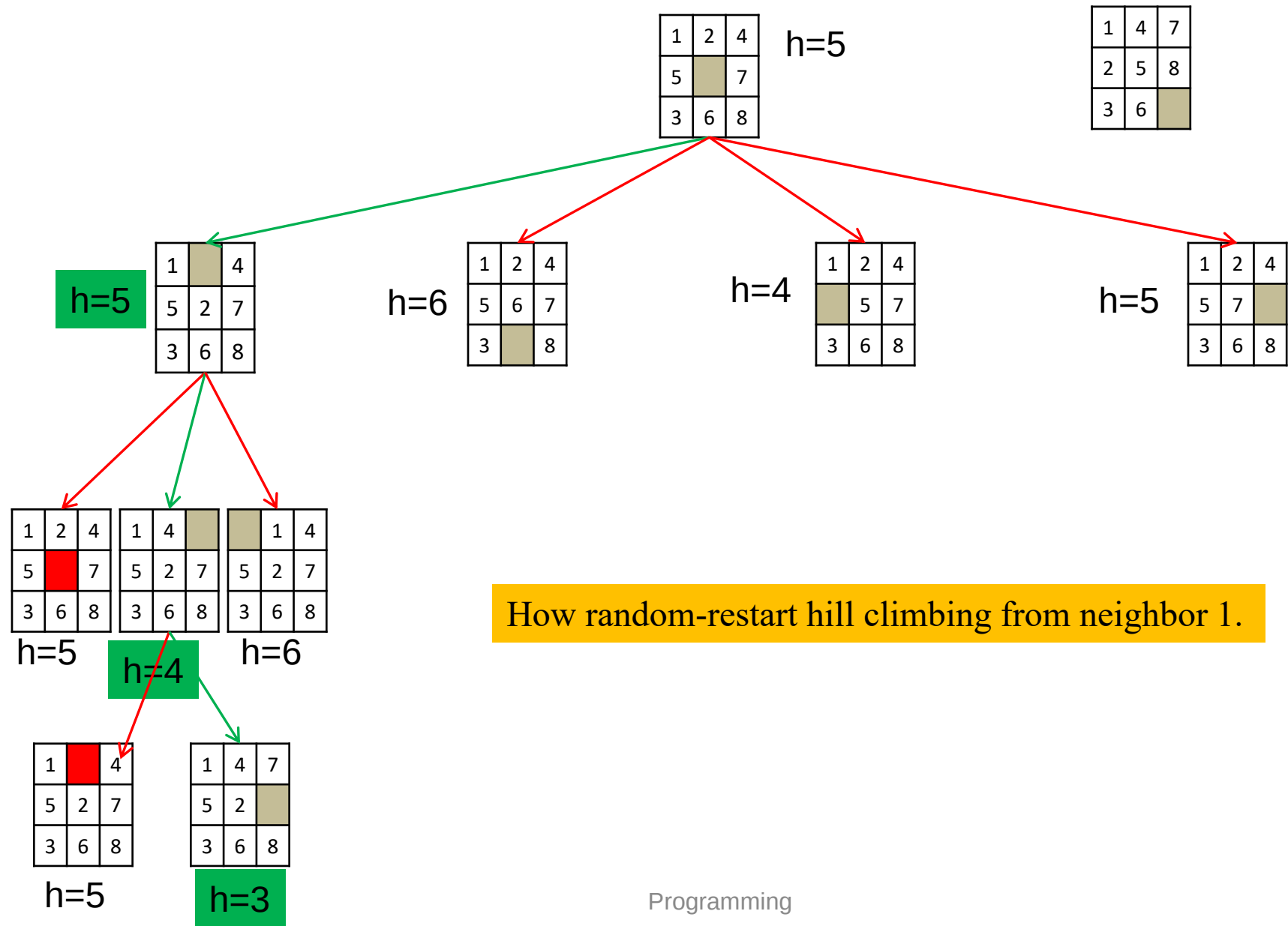
- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes, if $h(n)$ is admissible.

1. Understanding Hill-Climbing Algo

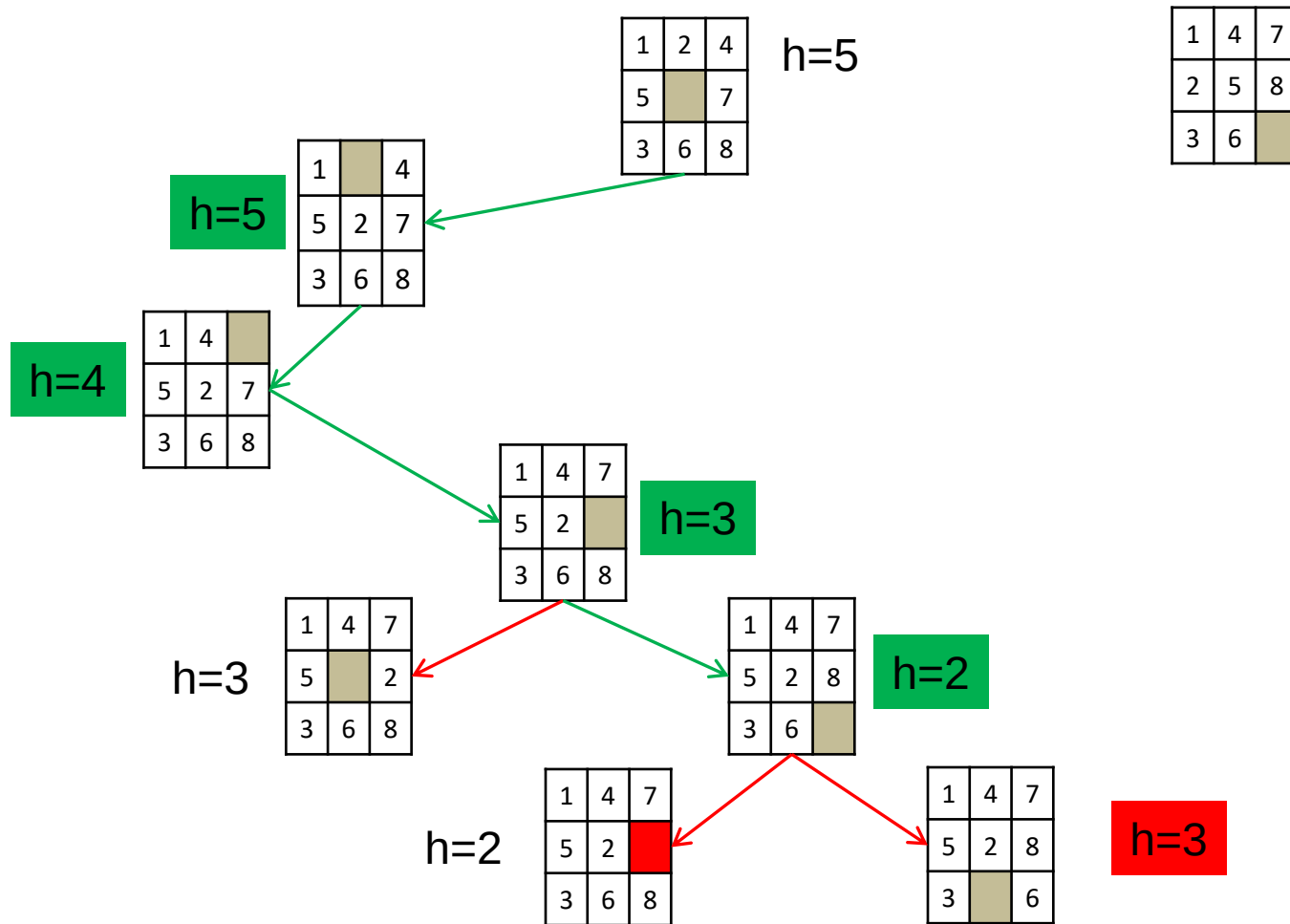


How random-restart hill climbing will work on the 8-puzzle.

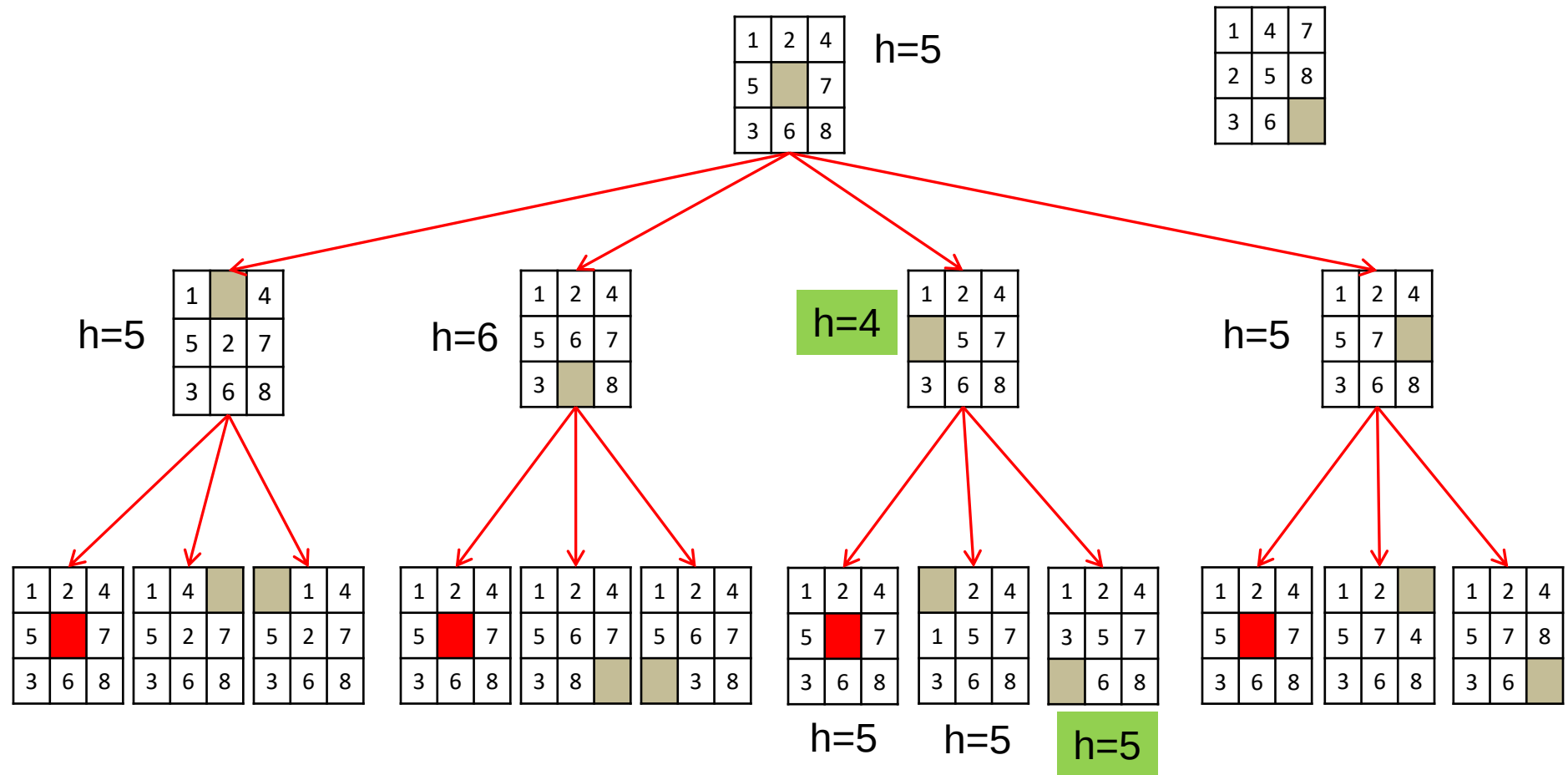
Understanding Hill-Climbing Algo



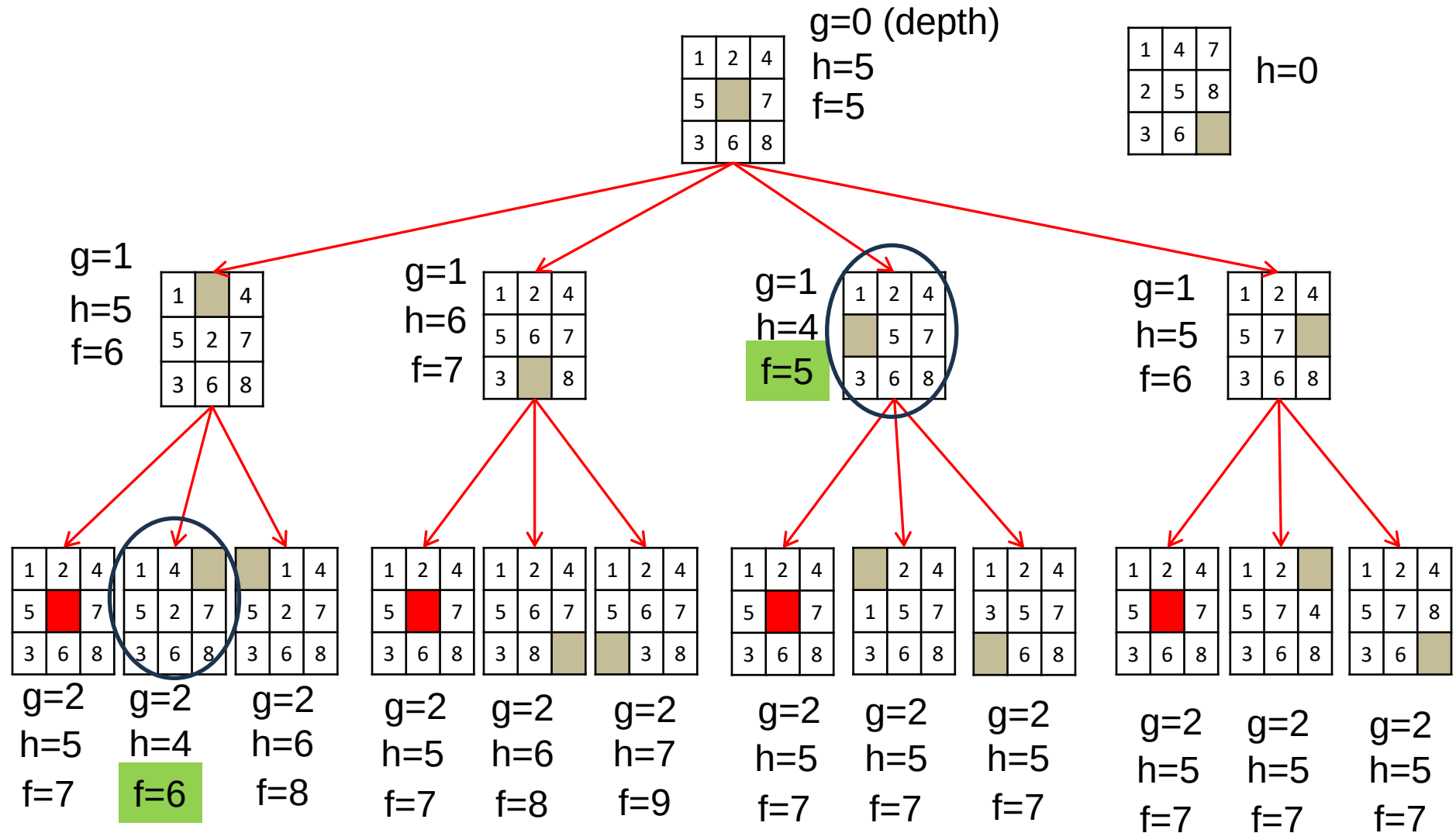
Understanding Hill-Climbing Algo



2. Understanding Greedy Best-First



Understanding A* Search



Comparing Uninformed/Blind Search

Evaluation of Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

Figure 27: Evaluation of Search Strategies.¹

Implementing Search Algorithms in python

Implementing Space State

1	2	4
5		7
3	6	8

1	4	7
2	5	8
3	6	

1		4
5	2	7
3	6	8

1	2	4
5	6	7
3		8

1	2	4
	5	7
3	6	8

1	2	4
5	7	
3	6	8

1	2	4
5		7
3	6	8

1	4	
5	2	7
3	6	8

	1	4
5	2	7
3	6	8

1	2	4
5		7
3	6	8

1	2	4
5	6	7
3	8	

1	2	4
5	6	7
	3	8

1	2	4
5		7
3	6	8

	2	4
1	5	7
3	6	8

1	2	4
3	5	7
	6	8

1	2	4
5		7
3	6	8

1	2	
5	7	4
3	6	8

1	2	4
5	7	8
3	6	

1. Display the Goal State.

1	4	7
2	5	8
3	6	0

```
goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]
```

```
def display_board(state):  
    print( "-----")  
    print( "| %i | %i | %i |" % (state[0], state[3], state[6]))  
    print( "-----")  
    print( "| %i | %i | %i |" % (state[1], state[4], state[7]))  
    print( "-----")  
    print( "| %i | %i | %i |" % (state[2], state[5], state[8]))  
    print( "-----")
```

```
display_board(goal_state)
```


2. Read the Initial State.

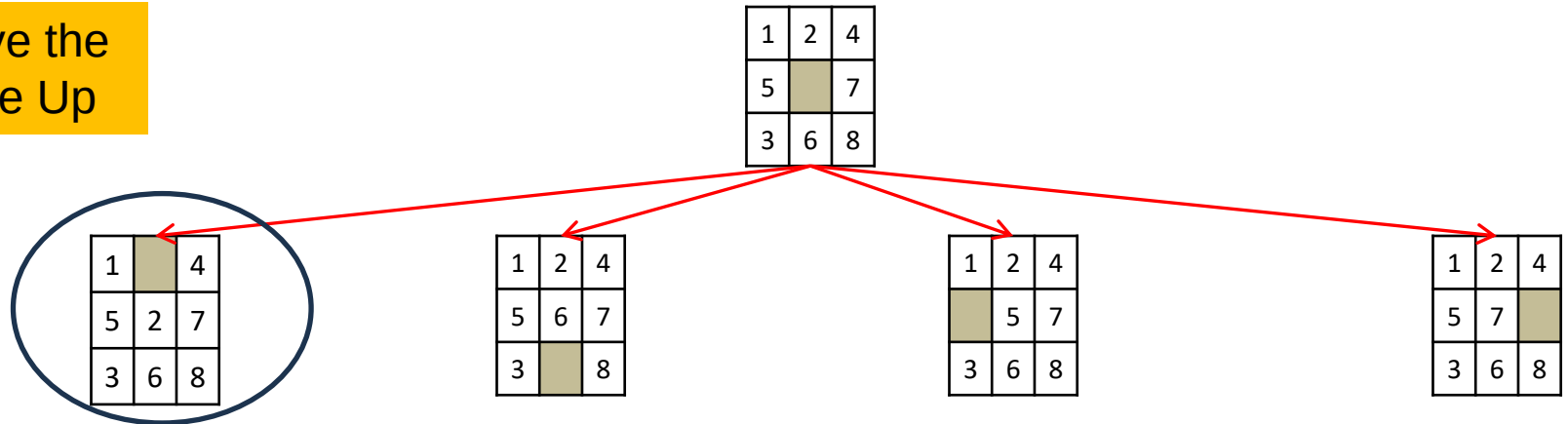
1	2	4
5	0	7
3	6	8

```
def readfile(filename):  
    f = open(filename)  
    data = f.read()  
    # Get rid of the newlines  
    data = data.strip("\n")  
    # Break the string into a list using  
    # a space as a separator.  
    data = data.split(" ")  
    state = []  
    for element in data:  
        state.append(int(element))  
    print('state: ', state)  
    return state
```

```
starting_state = readfile(r"state.txt")  
display_board(starting_state)
```

```
starting_state= [1, 5, 3, 2, 0, 6, 4, 7, 8]
```

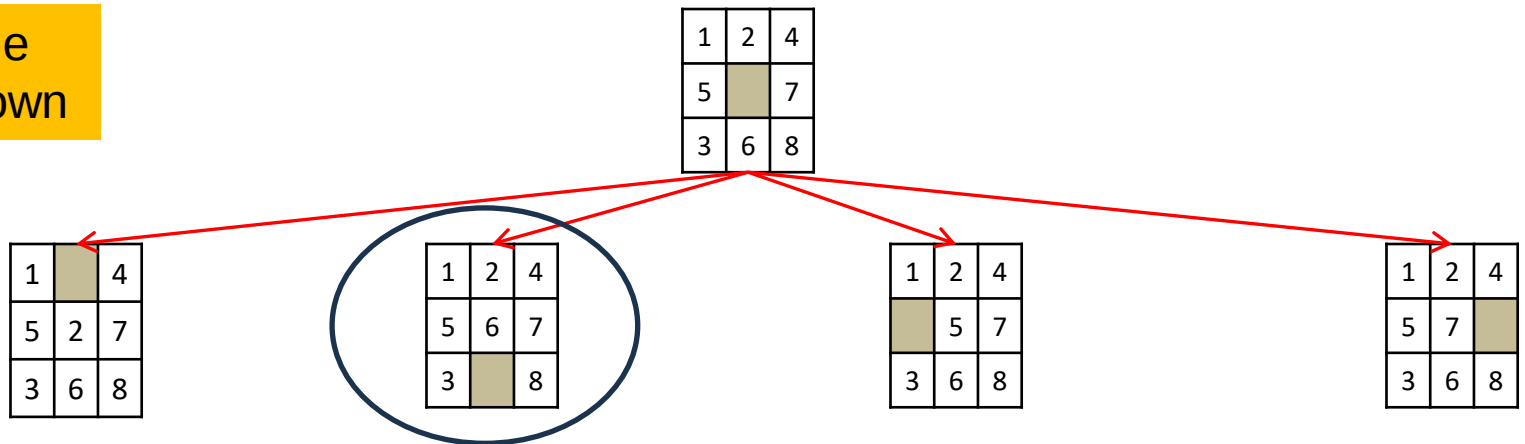
3.1 Move the blank tile Up



```
def move_up(state):  
    """Moves the blank tile up on the board.  
    Returns a new state as a list."""  
    # Perform an object copy  
    new_state = state[:]  
    index = new_state.index(0)  
    # Sanity check  
    if index not in [0, 3, 6]:  
        # Swap the values.  
        temp = new_state[index - 1]  
        new_state[index - 1] = new_state[index]  
        new_state[index] = temp  
        return new_state  
    else:  
        # Can't move, return None (Pythons NULL)  
        return None
```

```
display_board(move_up(starting_state))
```

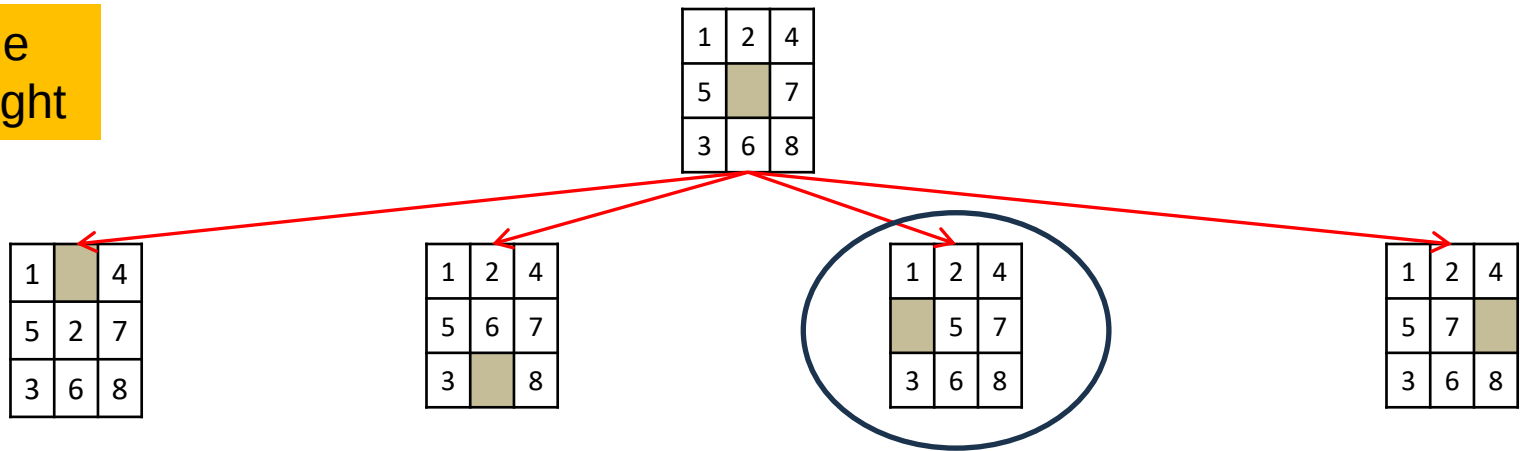
3.2 Move the blank tile down



```
def move_down(state):  
    """Moves the blank tile down on the board.  
    Returns a new state as a list."""  
    # Perform object copy  
    new_state = state[:]  
    index = new_state.index(0)  
    # Sanity check  
    if index not in [2, 5, 8]:  
        # Swap the values.  
        temp = new_state[index + 1]  
        new_state[index + 1] = new_state[index]  
        new_state[index] = temp  
        return new_state  
    else:  
        # Can't move, return None.  
        return None
```

```
display_board(move_down(starting_state))
```

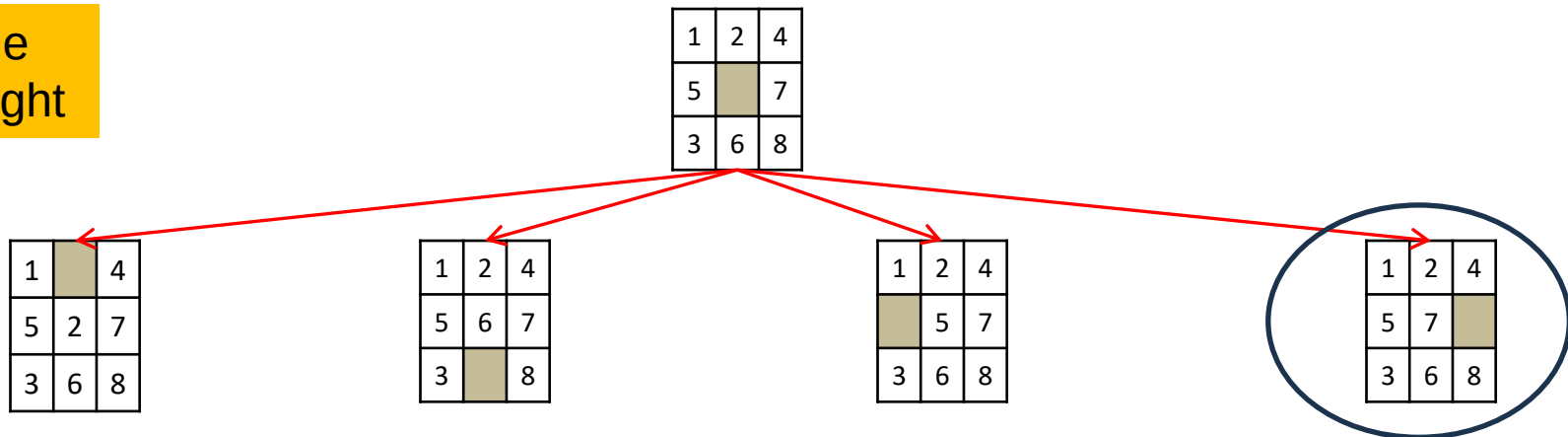
3.3 Move the blank tile Right



```
def move_left(state):  
    """Moves the blank tile left on the board.  
    Returns a new state as a list."""  
    new_state = state[:]  
    index = new_state.index(0)  
    # Sanity check  
    if index not in [0, 1, 2]:  
        # Swap the values.  
        temp = new_state[index - 3]  
        new_state[index - 3] = new_state[index]  
        new_state[index] = temp  
        return new_state  
    else:  
        # Can't move it, return None  
        return None
```

```
display_board(move_left(starting_state))
```

3.4 Move the blank tile Right



```
def move_right(state):  
    """Moves the blank tile right on the board.  
    Returns a new state as a list."""  
    # Performs an object copy. Python passes by reference.  
    new_state = state[:]  
    index = new_state.index(0)  
    # Sanity check  
    if index not in [6, 7, 8]:  
        # Swap the values.  
        temp = new_state[index + 3]  
        new_state[index + 3] = new_state[index]  
        new_state[index] = temp  
        return new_state  
    else:  
        # Can't move, return None  
        return None
```

```
display_board(move_right(starting_state))
```

4 Create a Node to keep information such as **state, parent, operator, depth, cost**.

```
def create_node(state, parent, operator, depth, cost):  
    return Node(state, parent, operator, depth, cost)  
  
# Node data structure  
class Node:  
    def __init__(self, state, parent, operator, depth, cost):  
        # Contains the state of the node  
        self.state = state  
        # Contains the node that generated this node  
        self.parent = parent  
        # Contains the operation that generated this node from  
        self.operator = operator  
        # Contains the depth of this node (parent.depth +1)  
        self.depth = depth  
        # Contains the path cost of this node from depth 0. No  
        self.cost = cost  
  
        self.heuristic=None
```

```
def bfs(start, goal):  
    """Create a node to Performs BFS search"""  
    # A List (can act as a queue) for the nodes.  
    goal=goal  
    start_node=create_node(start, None, None, 0, 0)  
    return start_node  
    pass
```

- The `__init__` method is a special method in Python classes, known as the constructor.
- It is automatically called when an instance of the class is created.

- ✓ When we define a method in a class, the first parameter of the method is always a reference to the instance of the class.
- ✓ This parameter is usually named `self`.
- ✓ This allows us to access the attributes and methods of the class in Python.

```
goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]  
starting_state= [1, 5, 3, 2, 0, 6, 4, 7, 8]  
starting_state = readfile(r"state.txt")  
result = bfs(starting_state, goal_state)  
if result.state == goal_state:  
    print('Start and Goal are same')  
else:  
    print('Initial Node:', result.state,  
          '\n Goal Node:', goal_state,  
          '\n You have to implement BFS')
```

5 Expand node

```
def expand_node(node):  
    """Returns a list of expanded nodes"""  
    expanded_nodes = []  
    expanded_nodes.append(create_node(move_up(node.state), node, "u", node.depth + 1, 0))  
    expanded_nodes.append(create_node(move_down(node.state), node, "d", node.depth + 1, 0))  
    expanded_nodes.append(create_node(move_left(node.state), node, "l", node.depth + 1, 0))  
    expanded_nodes.append(create_node(move_right(node.state), node, "r", node.depth + 1, 0))  
    # Filter the list and remove the nodes that are impossible (move function returned None)  
    expanded_nodes = [node for node in expanded_nodes if node.state != None] # List comprehension!  
    return expanded_nodes
```

```
starting_state = readfile(r"state.txt")  
all_neighbours = expand_node(create_node(starting_state, None, None, 0, 0))  
[(next_nodes.state, next_nodes.operator) for next_nodes in all_neighbours]
```

OR

Main method

```
def main():  
    starting_state = readfile(r"state.txt")  
    all_neighbours = expand_node(create_node(starting_state, None, None, 0, 0))  
    print([(next_nodes.state, next_nodes.operator) for next_nodes in all_neighbours])
```

starting_state = [1, 5, 3, 2, 0, 6, 4, 7, 8]

A python-isim. Basically if the file is being run execute the main() function.

```
if __name__ == "__main__":  
    main()
```

BFS Algorithm

```
def bfs(start, goal):  
    """Performs a breadth first search from  
    the start state to the goal"""  
    # A list (can act as a queue) for the nodes.  
    goal=goal  
    start_node=create_node(start, None, None, 0, 0)  
    fringe=[]  
    fringe.append(start_node)  
    current=fringe.pop(0)  
    path=[]  
    while(current.state!=goal):  
        fringe.extend(expand_node(current))  
        current=fringe.pop(0)  
    while(current.parent!=None):  
        path.insert(0, current.operator)  
        current=current.parent  
    return path  
pass
```


RUN the BFS Algorithm

```
goal_state = [1, 5, 3, 0, 2, 6, 4, 7, 8]

# Main method
def main():
    starting_state = readfile(r"state.txt")
    ### CHANGE to run bfs, dfs, Greedy or a_star
    result = bfs(starting_state, goal_state)

    if result == None:
        print( "No solution found")
    elif result == [None]:
        print( "Start node was the goal!")
    else:
        print(result)
        print(len(result), " moves")

# Execute the main() function.
if __name__ == "__main__":
    main()
```

Output->

```
state: [1, 5, 3, 2, 0, 6, 4, 7, 8]
['u']
1 moves
```

```

# Explore BFS code
def bfs(start, goal):
    """Performs a breadth first search from the
    start state to the goal"""
    # A List (can act as a queue) for the nodes
    goal=goal
    start_node=create_node(start, None, None, 0, 0)
    fringe=[]
    fringe.append(start_node)

    current=fringe.pop(0)
    path=[]
    while(current.state!=goal):
        fringe.extend(expand_node(current))
        # Check Explanded nodes
        for states in fringe:
            print(states.state)

        current=fringe.pop(0)

        # Check current node
        print('current node:', current.state)

    while(current.parent!=None):
        path.insert(0, current.operator)
        current=current.parent
    return path
pass

```

#Case-1: see the expanded nodes-
 goal_state = [1, 5, 3, 0, 2, 6, 4, 7, 8]

#Case 2: see the two moves-
 #goal_state = [1, 5, 3, 2, 7, 6, 4, 0, 8]

Case 3: Check more expansions
 #goal_state = [1, 5, 3, 2, 6, 8, 4, 7, 0]

Case 4: Check the Goal
 #goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]

UCS Algorithm

```
def uniform_cost(start,goal):
    start_node=create_node(start, None, None, 0, 0)
    fringe=[]
    path=[]
    fringe.append(start_node)
    current=fringe.pop(0)
    while(current.state!=goal):
        temp=expand_node(current)
        for item in temp:
            item.depth+=current.depth
            fringe.append(item)
        fringe.sort(key =lambda x: x.depth)
        current=fringe.pop(0)
    while(current.parent!=None):
        path.insert(0,current.operator)
        current=current.parent
    return path
```

***Change **bfs** to **uniform_cost** function:
result = **uniform_cost**(starting_state, goal_state)

Informed Search Algorithm

```
def h(state, goal):  
    dmatch=0  
    for i in range(0,9):  
        if state.state[i] != goal[i]:  
            dmatch+=1  
    state.heuristic=dmatch
```

Greedy Best-First Algorithm

```
def greedy(start,goal):
    start_node=create_node(start,None,None,0,0)
    fringe=[]
    path=[]
    fringe.append(start_node)
    current=fringe.pop(0)
    while(current.state!=goal):
        fringe.extend(expand_node(current))
        for item in fringe:
            h(item,goal)
        fringe.sort(key =lambda x: x.heuristic)
        current=fringe.pop(0)
    while(current.parent!=None):
        path.insert(0,current.operator)
        current=current.parent
    return path
```

***Change function name:
result = **greedy**(starting_state, goal_state)

A* Algorithm

```
def a_star(start, goal):
    start_node=create_node(start, None, None, 0, 0)
    fringe=[]
    path=[]
    fringe.append(start_node)
    current=fringe.pop(0)
    while(current.state!=goal):
        fringe.extend(expand_node(current))
        for item in fringe:
            h(item, goal)
            item.heuristic+=item.depth
        fringe.sort(key =lambda x: x.heuristic)
        current=fringe.pop(0)
    while(current.parent!=None):
        path.insert(0, current.operator)
        current=current.parent
    return path
```

***Change function name:
result = **a_star**(starting_state, goal_state)

Explore A* to see the states and values

```
def a_star(start, goal):
    start_node=create_node(start, None, None, 0, 0)
    fringe=[]
    path=[]
    fringe.append(start_node)
    current=fringe.pop(0)
    while(current.state!=goal):
        fringe.extend(expand_node(current))
        for item in fringe:
            h(item, goal)
            item.heuristic+=item.depth
        fringe.sort(key =lambda x: x.heuristic)
        # Check Expanded nodes
        print('Expanded nodes->')
        for states in fringe:
            print(states.state, states.heuristic, states.depth)
        #

        current=fringe.pop(0)
    while(current.parent!=None):
        path.insert(0, current.operator)
        current=current.parent
    return path
```

```
#Case-1: see the expanded nodes-
goal_state = [1, 5, 3, 0, 2, 6, 4, 7, 8]
#Case 2: see the two moves-
#goal_state = [1, 5, 3, 2, 7, 6, 4, 0, 8]
# Case 3: Check more expansions
#goal_state = [1, 5, 3, 2, 6, 8, 4, 7, 0]
# Case 4: Check the Goal
#goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]
```

Expanded nodes->

```
[1, 5, 3, 0, 2, 6, 4, 7, 8] 1 1
[1, 5, 3, 2, 6, 0, 4, 7, 8] 4 1
[1, 0, 3, 2, 5, 6, 4, 7, 8] 4 1
[1, 5, 3, 2, 7, 6, 4, 0, 8] 4 1
['u']
1 moves
```

```
def __init__(self, board):
    self.board = board
    self.goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

def find_zero(self):
    return self.board.index(0)

def possible_moves(self):
    zero_index = self.find_zero()
    moves = []
    if zero_index % 3 != 0: # Can move left
        moves.append(-1)
    if zero_index % 3 != 2: # Can move right
        moves.append(1)
    if zero_index // 3 != 0: # Can move up
        moves.append(-3)
    if zero_index // 3 != 2: # Can move down
        moves.append(3)
    return moves

def move(self, direction):
    zero_index = self.find_zero()
    new_index = zero_index + direction
    new_board = self.board[:]
    new_board[zero_index], new_board[new_index] = new_board[new_index], new_board[zero_index]
    return Puzzle(new_board)

def heuristic(self):
    return sum([1 if self.board[i] != self.goal[i] else 0 for i in range(9)])

def __str__(self):
    return '\n'.join([str(self.board[i:i+3]) for i in range(0, 9, 3)])
```


Hill-Climbing Algo

```
def hill_climbing(puzzle):
    current = puzzle
    while True:
        neighbors = [current.move(move) for move in current.possible_moves()]
        next_move = min(neighbors, key=lambda x: x.heuristic())
        if next_move.heuristic() >= current.heuristic():
            break
        current = next_move
    return current
```

```
# Example usage
initial_board = [1, 2, 3, 4, 5, 6, 0, 7, 8]
puzzle = Puzzle(initial_board)
solution = hill_climbing(puzzle)
print("Initial State:")
print(puzzle)
print("\nSolution State:")
print(solution)
```

Importance of List Comprehensions

```
import time

start = time.time()
squares = [x**2 for x in range(1000000)]
end = time.time()
print("List comprehension:", end - start)

start = time.time()
squares = []
for x in range(1000000):
    squares.append(x**2)
end = time.time()
print("For loop:", end - start)
```

```
List comprehension: 0.4214487075805664
For loop: 0.6247966289520264
```

Advanced List Comprehensions in Python

- <https://www.w3computing.com/articles/advanced-list-comprehensions-in-python/>
- <https://toxigon.com/python-advanced-list-comprehensions>
- <https://realpython.com/python-set-comprehension/>

Python: Production-Level Coding Practices

<https://medium.com/red-buffer/python-production-level-coding-practices-4c39246e0233>